

Operating Systems — Practice Exam (Example)

Instructor: Tsvi Cherny

TAs: Daniel Karalnik, Liam Tal

Time: 3 hours. **Total:** 100 points.

- **Part A — True/False:** 10 questions, 3 points each (30 pts). Answer all.
- **Part B — Multiple choice:** 4 questions, 5 points each (20 pts). Answer all.
- **Part C — Open questions:** 3 questions, 25 points each. **Answer any 2 of the 3** (50 pts).

For the open questions, show your work — partial credit is given per sub-part. Model solutions (with the point split) are at the end.

Disclaimer — Units and notation

Throughout this exam, **all storage-size units are base-2 (powers of 2)**. That is, $1 \text{ KB} = 1024 \text{ bytes}$, $1 \text{ MB} = 1024 \text{ KB}$, $1 \text{ GB} = 1024 \text{ MB}$, and so on. The prefixes **KB**, **MB**, **GB** as used here are therefore equivalent to the IEC binary prefixes **KiB**, **MiB**, **GiB** — we use the shorter forms (KB/MB/GB) purely as a convention, but they always mean the base-2 quantities.

Unit conversions:

Unit	Equals	In bytes	Power of 2
1 byte	8 bits	—	2^3 bits
1 KB (= 1 KiB)	1024 bytes	1,024 B	2^{10} B
1 MB (= 1 MiB)	1024 KB	1,048,576 B	2^{20} B
1 GB (= 1 GiB)	1024 MB	1,073,741,824 B	2^{30} B

So, for example, $1 \text{ GB} = 1024 \text{ MB} = 1024 \times 1024 \text{ KB} = 1024 \times 1024 \times 1024 \text{ bytes} = 2^{30} \text{ bytes}$.

Part A — True / False (3 points each)

For each statement, **circle** the correct answer (**True / False**) shown at the end.

A1. A plain test-and-set spinlock guarantees that every waiting thread eventually acquires the lock.

True / False

A2. `setjmp` returns 0 when it first saves the context, and a non-zero value when control later returns to it via `longjmp`.

True / False

A3. Round Robin with a time quantum larger than every job's CPU burst behaves exactly like FCFS.

True / False

A4. A hierarchical (multi-level) page table always consumes more total physical memory than a single-level page table covering the same virtual address space.

True / False

A5. RAID 5 survives any single disk failure by reconstructing the lost data from parity, but cannot survive two simultaneous disk failures.

True / False

A6. Under lazy binding, an external function's real address is resolved when the program is loaded, before `main` runs.

True / False

A7. Recall that `cond_wait` atomically releases the mutex and blocks the calling thread until the condition variable is signalled, then re-acquires the mutex before returning. A thread returning from `cond_wait` must re-check the predicate (using `while`, not `if`) before proceeding, because a wakeup does not by itself guarantee that the predicate now holds.

True / False

A8. A divide-by-zero and a `write()` call both transfer control to the kernel, but the divide-by-zero is an exception (unexpected) while `write()` is a system call (intentional).

True / False

A9. On a context switch between two processes, the operating system must always clear the entire TLB.

True / False

A10. Every ELF executable must contain a function named `main`, which the OS uses as the entry point.

True / False

Part B — Multiple choice (5 points each)

Circle the single best answer.

B1. A function calls `setjmp(buf)` and then **returns** to its caller. Later, `main` calls `longjmp(buf, 1)`. What is the result?

- A. Execution safely resumes inside that function
- B. Undefined behavior — jumping into a stack frame that no longer exists
- C. `setjmp` returns 0
- D. An infinite loop

B2. Using the burst estimator $\tau_{n+1} = \alpha \cdot \tau_n + (1 - \alpha) \cdot t_n$ with $\alpha = 0.5$, current estimate $\tau_n = 8 \text{ ms}$, and last actual burst $t_n = 4 \text{ ms}$, the next estimate is:

- A. 4
- B. 6
- C. 8
- D. 12

B3. An ext-style inode has 12 direct pointers, 1 single-indirect, 1 double-indirect, and 1 triple-indirect pointer. Block size 4 KB, pointer size 4 bytes (1024 pointers per block). For a **100 MB** file, how much space do the **indirect (pointer) blocks** consume?

- A. 8 KB
- B. 96 KB

C. 100 KB

D. 104 KB

B4. Under lazy binding, what does a function's GOT entry point to before that function is first called?

A. the real function's address

B. NULL

C. back into the PLT stub / dynamic-linker resolver, which patches the GOT on the first call

D. main

Part C — Open questions (25 points each — answer any 2 of 3)

C1. Thread interleaving under a lock

A global integer x starts at 0, shared by two threads. A single mutex m protects x . Each numbered statement below is its own critical section (`lock(m); <statement>; unlock(m);`).

- Thread A: A1: $x = x + 1$; then A2: $x = x * 2$;
- Thread B: B: $x = x + 3$;

The three critical sections run in some order, with the only constraint that A1 runs before A2. **Note:** A1 and A2 are *separate* critical sections — m is released between them, so another thread's critical section may run in between.

a) (6 pts) List every possible final value of x .

b) (3 pts) Give an ordering of A1, A2, B that produces the **smallest** final value.

c) (6 pts) A different program uses two mutexes:

- Thread 1: `lock(L1); lock(L2); ...; unlock(L2); unlock(L1);`
- Thread 2: `lock(L2); lock(L1); ...; unlock(L1); unlock(L2);`

(i) Can this deadlock? (yes / no)

(ii) Which rule prevents it?

A. Every thread acquires the two locks in the same global order

B. Bigger critical sections

C. Unlock in acquisition order

D. Add a third lock

d) (10 pts) Back to the single-mutex program, add a third thread C: $x = x - 1$; (its own critical section, no ordering constraint vs A or B). With all four critical sections interleaved (A1 still before A2), how many **distinct** final values of x are possible? List them.

C2. Copy-on-write with threads and `fork()`

A process maps a 4-page region with `mmap(..., PROT_READ|PROT_WRITE, MAP_PRIVATE, fd, 0)`. Page states: **Not resident** (PTE absent), **Resident-clean** (present, read-only, shared with the file page), **Resident-dirty** (present, writable, private copy made). A read of a Not-resident page faults it to Resident-clean. A write faults it to Resident-dirty

(making a private copy). Writing a Resident-clean page faults to Resident-dirty. Anything on a Resident-dirty page is free.

a) (12 pts) Threads T_1 , T_2 share the page table (a fault by one makes the page resident for both). For each step, give Fault? (Y/N), Private copy this step? (Y/N), and the page's resulting state.

Step	Access	Fault?	Copy?	State after
1	T1 reads P0			
2	T2 reads P0			
3	T1 writes P0			
4	T2 writes P1			
5	T1 reads P2			
6	T2 writes P2			
7	T1 reads P1			

b) (13 pts) Start fresh (notation: **W P0** means *write to page P0*, **R P0** means *read from page P0*). The process does **W P0**, then **R P0**, then calls `fork()` (which write-protects every currently-resident page in *both* parent and child). Then: **Child W P0**, **Parent R P0**, **Parent W P0**. For each step give Fault? (Y/N), Copy made? (Y/N), and P0's state in parent and child.

Step	Who	Access	Fault?	Copy?	Parent P0	Child P0
1	Proc	W P0				—
2	Proc	R P0				—
3	—	fork()	—	—		
4	Child	W P0				
5	Parent	R P0				
6	Parent	W P0				

C3. MLFQ vs RSDL

Three threads arrive at $t = 0$ on one CPU (context switches free), entering the run queue in order T_{cpu} , T_{int} , T_{mix} . Profiles (ms):

- T_{cpu} : CPU 7
- T_{int} : CPU 1, IO 3, CPU 1, IO 3, CPU 1, IO 3, CPU 1
- T_{mix} : CPU 3, IO 1, CPU 3

Scheduler A — MLFQ. Queues $q_0 > q_1 > q_2$ with quanta 1, 2, 4 ms. New threads start in q_0 . Using a full quantum on the CPU demotes a thread one level (q_2 is the floor). Blocking for I/O before the quantum expires **keeps the thread at its level** (it returns to that level when I/O completes). The highest non-empty queue runs, round-robin within a level. No priority boost.

Alter question. This MLFQ keeps a blocking thread at its level (in the version from class, blocking **promotes**). Apply exactly the rules above.

Scheduler B — RSDL. Levels 0..3, each thread gets a **2 ms quota per level**, and all start at level 0. The highest occupied level runs round-robin in **1 ms turns**, and each ms decrements the running thread's quota at its level. When

that quota hits 0 the thread **descends** one level with a fresh 2 ms quota. A thread that blocks for I/O keeps its level and remaining quota. When every ready thread has reached the bottom level, all reset to level 0.

a) (8 pts) What is the **finish order** under MLFQ?

b) (8 pts) What is the **finish order** under RSDL?

c) (9 pts) Fill the per-ms **MLFQ** Gantt chart for $t = 0..16$ (write $C = T_{cpu}$, $I = T_{int}$, $M = T_{mix}$ in each slot).

t: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16

Solution

(Correct True/False answer shown in **bold**.)

Part A

- A1. True / **False** — TAS is not starvation-free.
- A2. **True** / False
- A3. **True** / False
- A4. True / **False** — sparse address spaces omit lower-level tables, using less memory.
- A5. **True** / False
- A6. True / **False** — lazy binding resolves on the first call, not at load.
- A7. **True** / False
- A8. **True** / False
- A9. True / **False** — PCID/ASID-tagged TLBs need no clearing.
- A10. True / **False** — the entry point is `_start`, and `main` is called by the C runtime.

Part B

- B1 → B. Once the function that called `setjmp` has returned, its stack frame is dead, so the environment saved in `buf` points at memory that is no longer valid. Jumping back into it is undefined behavior. `longjmp` is only safe while the function that called `setjmp` is still active on the stack.
- B2 → B. $0.5 \cdot 4 + 0.5 \cdot 8 = 6$.
- B3 → D. **104 KB**. Count only the pointer blocks needed to map all the data blocks.
 - Data blocks: $100 \text{ MB} / 4 \text{ KB} = 25,600$.
 - The **12 direct** pointers map the first 12 data blocks and need **no** pointer block.
 - The **single-indirect** pointer maps the next 1024 blocks, and costs **1** pointer block (the block that holds those 1024 pointers).
 - Blocks still unmapped: $25,600 - 12 - 1024 = 24,564$. These go through the **double-indirect** pointer.
 - The double-indirect pointer points to **1** top-level block, and that block points to “leaf” pointer blocks that each map 1024 data blocks. Leaves needed: $\lceil 24,564 / 1024 \rceil = 24$.
 - Total pointer blocks: $1 \text{ (single)} + 1 \text{ (double top-level)} + 24 \text{ (leaves)} = 26$. Overhead: $26 \times 4 \text{ KB} = \mathbf{104 \text{ KB}}$. (The triple-indirect pointer is never reached, so it costs nothing.)
- B4 → C. Initially the GOT entry routes back to the resolver stub. After the first call it holds the real address.

Part C1 — Interleaving (25 pts: a) 6, b) 3, c) 6, d) 10)

- a) {5, 8}. B, A1, A2 → 8. A1, B, A2 → 8. A1, A2, B → 5.
- b) A1, A2, B → 0 → 1 → 2 → 5 (smallest = 5).
- c) (i) Yes (circular wait). (ii) A — one consistent global lock order.
- d) 4 distinct values: {3, 4, 6, 7} (enumerate the 12 orderings of A1, A2, B, C with A1 before A2).

Part C2 — Copy-on-write (25 pts: a) 12, b) 13)

a)

Step	Access	Fault?	Copy?	State after
1	T1 R P0	Y	N	Resident-clean
2	T2 R P0	N	N	Resident-clean
3	T1 W P0	Y	Y	Resident-dirty
4	T2 W P1	Y	Y	Resident-dirty
5	T1 R P2	Y	N	Resident-clean
6	T2 W P2	Y	Y	Resident-dirty
7	T1 R P1	N	N	Resident-dirty

b)

Step	Access	Fault?	Copy?	Parent P0	Child P0
1	Proc W P0	Y	Y	Resident-dirty	—
2	Proc R P0	N	N	Resident-dirty	—
3	fork()	—	—	write-protected	write-protected
4	Child W P0	Y	Y	write-protected	private dirty
5	Parent R P0	N	N	write-protected	private dirty
6	Parent W P0	Y	N	private dirty	private dirty

(Step 6 is the discriminator: the parent's page now has reference count 1 — the child already copied — so the write just makes it writable with **no** copy.)

Part C3 — MLFQ vs RSDL (25 pts: a) 8, b) 8, c) 9)

- a) MLFQ finish order: T_int → T_cpu → T_mix (finishes at t = 14, 16, 17).
- b) RSDL finish order: T_int → T_mix → T_cpu (finishes at t = 14, 16, 17).
- c) MLFQ Gantt:

t: C I M C C I M M C I M M C I C C M

(T_int sleeps through contention so it always gets q0 promptly and finishes first under both. The split is T_cpu vs T_mix: under MLFQ the always-ready T_cpu soaks up the CPU during the others' I/O and finishes one tick ahead of T_mix. Under RSDL's strict 1 ms round-robin it cannot, so the shorter T_mix finishes first.)

